Kelompok Mini Project KB
Nama Anggota:

1. Destama Rohman Nuswantoro (21507334088)
2. Muhammad Ibrahim Musyaffa (22507334051)
3. Syahrijal Aulia Rasyidin (22507334058)

---

1. Data foto yang dipakai sebagai data latih
   Data foto untuk model ada di dalam file folder ZIP yang dikirimkan dan dinamai dengan Known_faces

2. rencana model deep learning yang dipakai, dengan disertai alasan pemilihannya
   Untuk penggunaan face detection yang menggunakan Yunet (CNN), dan face recognition yang menggunakan CNN Resnet.

   Alasan Pemilihan Model:

a. Model CNN YuNet dipilih karena model ini didesain untuk mendeteksi wajah dengan akurasi tinggi dan sangat efisien dalam hal pemrosesan wajah, bayangkan saja YuNet dapat mendeteksi 8 wajah dalam waktu 5.09ms dibandingkan dengan face detection lain seperti Cascade Classifier yang mendeteksi 8 wajah dengan waktu 25.81ms. Karena inilah YuNet dipilih sebagai model pendeteksi wajah

b. Model CNN di Face_recognition dipilih karena merupakan model yang dirancang untuk pengenalan wajah dengan akurasi cukup tinggi dan mudah digunakan. Model ini mampu menghasilkan face encoding yang akurat dan melakukan pencocokan wajah secara efisien menggunakan hasil pelatihan model gambar kita dengan model milik Face_recognition yang nantinya menghasilkan hasil anotasi di bagian “encodings”. Selain itu, Face_recognition dapat digabungkan dengan YuNet sehingga memungkinkan face detection lalu dilanjutkan dengan face recognition dan dapat dijalan secara real time melalui webcam .

3. Tahap preprocessing yang akan dipakai, disertai dengan alasannya
   Pre processing Deep Learning:

   1). Persiapan Data:

- Pengumpulan Data: Data diambil dari masing-masing anggota kelompok, 15 data per-anggota.
- Anotasi Data: Anotasi dilakukan dengan memanfaatkan model pada library face_recognition, sehingga data foto yang dikumpulkan telah di anotasi secara otomatis
- Preprocessing: Seluruh data hasil foto kemudian akan disimpan kedalam file bernama “known_faces.pkl” , yang kemudian akan dipanggil di kode utama.

2). Model untuk Deteksi Wajah:

- Pemilihan Model: YuNet (CNN) dikenal untuk kemampuan deteksi wajah yang efisien dan akurat.
- Implementasi Model: Menggunakan model yang telah di training oleh Yunet untuk mendeteksi wajah serta berformat ONNX dengan nama model yaitu “face_detection_yunet_2023mar.onnx”. Kemudian model yang telah didapat digunakan untuk melakukan prediksi wajah melalui webcam ataupun gambar

3). Model untuk Pengenalan Wajah:

- Pemilihan Model: `face_recognition` menggunakan CNN ResNet yang sangat baik dalam menangkap fitur wajah yang mendetail.
- Implementasi Model: Gunakan library `face_recognition` yang berbasis pada ResNet untuk melakukan pengenalan wajah.
- Pelatihan Model: Model pada data foto akan di anotasi dengan model milik face_recognition di kode precompute.py. Hal ini agar dapat mempercepat proses inference di kode utama.

4). Alur penggunaan model:

- Open webcam → Face detection (Yunet) → Deteksi wajah → Resize gambar → convert gambar ke RGB → Output deteksi wajah → Face recognition (face_recognition) →[Bandingkan dengan model wajah] → output face recog

4. File coding (jika sudah ada)
   Kode utama dapat dilihat pada file face_reco.py atau bisa juga melihat dibawah ini

   dengan contoh penggunaan yaitu
   1. python face_reco.py -w → untuk input webcam

2. python face_reco.py -i path/pada/image → untuk input gambar

```
import cv2 as cv
import numpy as np
import face_recognition
import pickle
import argparse
import os
from yunet import YuNet

# Function to resize and pad image to match model's expected
input size
def resize_with_padding(image, target_size=(320, 320)):
    h, w = image.shape[:2]
    scale = min(target_size[0] / h, target_size[1] / w)
    new_w = int(w * scale)
    new_h = int(h * scale)
    resized_image = cv.resize(image, (new_w, new_h))

    padded_image = np.zeros((target_size[0], target_size[1], 3),
dtype=np.uint8)
    top = (target_size[0] - new_h) // 2
    bottom = target_size[0] - new_h - top
    left = (target_size[1] - new_w) // 2
    right = target_size[1] - new_w - left
    padded_image[top:top + new_h, left:left + new_w] =
resized_image

    return padded_image

# process_image function
def process_image(image_path, known_face_encodings,
known_face_names, model):
    image = cv.imread(image_path)
    if image is None:
        print(f"Error: Could not read the image file
'{image_path}'.")
        return

    # Resize the image with padding to fit the model input size
    resized_image = resize_with_padding(image, target_size=(320,
320))
```

```
    # Perform face detection and recognition
    results = model.infer(resized_image)
    output_image = visualize_and_recognize(resized_image,
results, known_face_encodings, known_face_names)

    return output_image  # Only return the processed image


# Function to process webcam input
def process_webcam(known_face_encodings, known_face_names,
model):
    cap = cv.VideoCapture(0)
    if not cap.isOpened():
        print("Error: Could not access the webcam.")
        return

    w = int(cap.get(cv.CAP_PROP_FRAME_WIDTH))
    h = int(cap.get(cv.CAP_PROP_FRAME_HEIGHT))
    model.setInputSize([w, h])

    tm = cv.TickMeter()
    while cv.waitKey(1) < 0:
        hasFrame, frame = cap.read()
        if not hasFrame:
            print('No frames grabbed!')
            break

        tm.start()
        results = model.infer(frame)
        tm.stop()

        frame = visualize_and_recognize(frame, results,
known_face_encodings, known_face_names, fps=tm.getFPS())
        cv.imshow('Webcam Detection & Recognition', frame)
        tm.reset()

    cap.release()
    cv.destroyAllWindows()

# Visualize and recognize faces in the frame
def visualize_and_recognize(image, results,
```

```
known_face_encodings, known_face_names, box_color=(0, 255, 0),
text_color=(0, 0, 255), fps=None):
    output = image.copy()

    if fps is not None:
        cv.putText(output, 'FPS: {:.2f}'.format(fps), (10, 30),
cv.FONT_HERSHEY_SIMPLEX, 1, text_color, 2)

    for det in results:
        bbox = det[0:4].astype(np.int32)
        conf = det[-1]

        # Extract face for recognition processing
        top, left, bottom, right = bbox[1], bbox[0], bbox[1] +
bbox[3], bbox[0] + bbox[2]

        # Ensure the coordinates are within bounds of the image
        top = max(0, top)
        left = max(0, left)
        bottom = min(image.shape[0], bottom)
        right = min(image.shape[1], right)

        face_image = image[top:bottom, left:right]

        # Check if face_image is valid (not empty)
        if face_image.size > 0:
            # Convert the face image to RGB for face_recognition
processing
            face_image_rgb = cv.cvtColor(face_image,
cv.COLOR_BGR2RGB)
            face_encodings =
face_recognition.face_encodings(face_image_rgb)

            if face_encodings:
                face_encoding = face_encodings[0]
                matches =
face_recognition.compare_faces(known_face_encodings,
face_encoding)
                name = "Unknown"
                if matches:
                    face_distances =
face_recognition.face_distance(known_face_encodings,
```

```
face_encoding)
                    best_match_index = np.argmin(face_distances)
                    if matches[best_match_index]:
                        name =
known_face_names[best_match_index]

                # Draw label and confidence
                text = f'{name} ({conf:.2f})'
                text_color = (0, 255, 0) if name != "Unknown"
else (0, 0, 255)
            else:
                # No face encoding found, label as unknown with
detection confidence
                text = f'Unknown ({conf:.2f})'

            # Draw rectangle around the face
            cv.rectangle(output, (left, top), (right, bottom),
box_color, 2)

            # Draw label
            cv.putText(output, text, (left, top - 10),
cv.FONT_HERSHEY_DUPLEX, 0.5, text_color)

    return output

def main():
    # Argument parser for input selection
    parser = argparse.ArgumentParser(description="Face
Recognition with YuNet and face_recognition")
    parser.add_argument('-i', '--input', type=str, help='Input
image file path for face recognition')
    parser.add_argument('-w', '--webcam', action='store_true',
help='Use webcam for face recognition')

    args = parser.parse_args()

    # Load precomputed face encodings
    with open("known_faces.pkl", "rb") as f:
        known_face_encodings, known_face_names = pickle.load(f)

    # Initialize YuNet with model path and parameters
    model = YuNet(
```

```
        modelPath='face_detection_yunet_2023mar.onnx',
        inputSize=[320, 320],
        confThreshold=0.9,
        nmsThreshold=0.3,
        topK=5000,
        backendId=cv.dnn.DNN_BACKEND_OPENCV,
        targetId=cv.dnn.DNN_TARGET_CPU
    )

    if args.input:
        # Process the input image
        result_image = process_image(args.input,
known_face_encodings, known_face_names, model)
        if result_image is not None:
            cv.imshow('Face Recognition - Image', result_image)
# Display the result
            cv.waitKey(0)
            cv.destroyAllWindows()
        else:
            print("Failed to process the image.")
    elif args.webcam:
        # Process webcam input
        process_webcam(known_face_encodings, known_face_names,
model)
    else:
        print("Please provide either an image input with -i or
webcam input with -w")

if __name__ == "__main__":
    main()
```